

*Communication ports definition of
micro controller and fuel dipsenser*



ZHEJIANG GENUINE MACHINE CO., LTD.

1. The code of communication data adopt compressed BCD code, the baud rate is 4800pts, check and(CRC8).

2. The communication port of fuel dispenser adopt RS485, semiduplex (management system initiative)

Frame format:

Start bit	DATA0	1	2	3	4	5	6	7	Stop bit
-----------	-------	---	---	---	---	---	---	---	----------

3. Communication data packet format:

Master: header + device address + function code + data origin address + data length+ check and (CRC8)

Slave: header + device address + function code + data length + data 0...N + check and (CRC8)

Header: 0xA5 1 byte

Device address: 0x01...0x99 1 byte

Function code: read 0x03 1 byte

read event 0x0c 1byte

write 0x10 1 byte

Data origin address: read/write data origin address 1 byte

Data length: the sum of data 0 ... N 1 byte

Check and: 1.CRC8(device address + function code + data origin address + data length) 1 byte

2.CRC8(device address + function code + data length + data 0...N) 1byte

4. Command

4.1 To check the status of device:

(e.g.) :To check device address no.01, read function code 0x03, data origin address 0x00, data length 0x01, CRC8.

Master 0xA5 + 0x01 + 0x03 + 0x00 + 0x01 + CRC8.

Slave:

1. device idle

0xA5 + 0x01 + 0x03 + 0x01 + 0x00 + CRC8;

2. device require to start as "litre"

0xA5 + 0x01 + 0x03 + 0x01 + 0x01 + CRC8;

3. device require to start as "sale"

0xA5 + 0x01 + 0x03 + 0x01 + 0x02 + CRC8;

4. device required to stop

0xA5 + 0x01 + 0x03 + 0x01 + 0x03 + CRC8;

5. device busy

0xA5 + 0x01 + 0x03 + 0x01 + 0x04 + CRC8;

6. device does not put nozzle back

0xA5 + 0x01 + 0x03 + 0x01 + 0x05 + CRC8;

7. device status after restart

0xA5 + 0x01 + 0x03 + 0x01 + 0x06 + CRC8;

8. if information is wrong

0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04 / 0x05/ 0x06 / 0x07 + CRC8;

4.2 To get the requirement of starting and then to check the preset value

(e.g.) : Check device address no.01, read function code 0x03, data origin address 0x10(volume) or 0x14(sale), data length 0x04, CRC8.

4.2.1

Master: 0xA5 + 0x01 + 0x03 + 0x10 + 0x04 + CRC8.

Slave:

1. device start fueling as default 9999.99 (liters) without preset value.
 $0xA5 + 0x01 + 0x03 + 0x04 + 0x99 + 0x99 + 0x99 + 0x00 + CRC8;$
2. device start fueling as preset value 123.45 liters
 $0xA5 + 0x01 + 0x03 + 0x04 + 0x01 + 0x23 + 0x45 + 0x00 + CRC8;$
3. if information is wrong
 $0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + CRC8;$

4.2.2

Master: 0xA5 + 0x01 + 0x03 + 0x14 + 0x04 + CRC8.

Slave:

1. device start fueling as default 999999.99 (sales) without preset value.
 $0xA5 + 0x01 + 0x03 + 0x04 + 0x99 + 0x99 + 0x99 + 0x99 + CRC8;$
2. device start fueling as preset value 123.45 (sales)
 $0xA5 + 0x01 + 0x03 + 0x04 + 0x00 + 0x01 + 0x23 + 0x45 + CRC8;$
3. if information is wrong
 $0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + CRC8;$

4.3 device start fueling directly

(e.g.) : Check device address no.01, write function code 0x10, data origin address 0x10(volume) or 0x14(sale), data length 0x04, CRC8.

4.3.1

Master:

Start fueling as preset 236.85 liters

$0xA5 + 0x01 + 0x10 + 0x10 + 0x04 + 0x02 + 0x36 + 0x85 + 0x00 + CRC8.$

Slave:

1. If information is correct
 $0xA5 + 0x01 + 0x10 + 0x10 + 0x04 + CRC8;$
2. If information is wrong
 $0xA5 + 0x01 + 0x90 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + CRC8;$

4.3.2

Master:

Start fueling as preset 123456.78 (sales)

$0xA5 + 0x01 + 0x10 + 0x14 + 0x04 + 0x12 + 0x34 + 0x56 + 0x78 + CRC8.$

Slave:

1. If information is correct
 $0xA5 + 0x01 + 0x10 + 0x14 + 0x04 + CRC8;$
2. If information is wrong
 $0xA5 + 0x01 + 0x90 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + CRC8;$

4.4 To check real-time data of fueling

(e.g.): Check device address no.01, read function code 0x03, data origin address 0x20, data length 0x08, CRC8.

Master:

$0xA5 + 0x01 + 0x03 + 0x20 + 0x08 + \text{CRC8}$.

Slave:

1. The real-time data of fueling, volume: 58.37L; sale: 123456.78

$0xA5 + 0x01 + 0x03 + 0x08 + 0x00 + 0x58 + 0x37 + 0x00 + 0x12 + 0x34 + 0x56 + 0x78 + \text{CRC8}$;

2. If information is wrong

$0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + \text{CRC8}$;

Remark: when receiving error code 0x03, the device stop working and we need do as item 4.5

4.5 Check the data when device stop working

(e.g.): Check device address no.01, read function code 0x03, data origin address 0x30, data length 0x08, CRC8.

Master:

$0xA5 + 0x01 + 0x03 + 0x30 + 0x08 + \text{CRC8}$.

Slave:

1. The current data of fueling, volume: 58.37L; sale: 123456.78.

$0xA5 + 0x01 + 0x03 + 0x08 + 0x00 + 0x58 + 0x37 + 0x00 + 0x12 + 0x34 + 0x56 + 0x78 + \text{CRC8}$;

2. If information is wrong

$0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + \text{CRC8}$;

4.6 To set device status as "control (0x00)" or "monitor (0x01)",

(e.g.): Check device address no.01, write function code 0x10, data origin address 0x5A, data length 0x01, CRC8.

Master:

$0xA5 + 0x01 + 0x10 + 0x5A + 0x01 + 0x00 + \text{CRC8}$.

Slave:

1. If information is correct

$0xA5 + 0x01 + 0x10 + 0x5A + 0x01 + \text{CRC8}$;

2. If information is wrong

$0xA5 + 0x01 + 0x90 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + \text{CRC8}$;

4.7 To set unit price

(e.g.): Check device address no.01, write function code 0x10, data origin address 0x55, data length 0x03, CRC8.

For example, unit price: 236.85/litre

Master:

$0xA5 + 0x01 + 0x10 + 0x55 + 0x03 + 0x02 + 0x36 + 0x85 + \text{CRC8}$.

Slave:

1. If information is correct

$0xA5 + 0x01 + 0x10 + 0x55 + 0x03 + \text{CRC8}$;

2. If information is wrong

$0xA5 + 0x01 + 0x90 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + \text{CRC8}$;

4.8 To check unit price

(e.g.): Check device address no.01, read function code 0x03, data origin address 0x55, data length 0x04, CRC8.

Master:

0xA5 + 0x01 + 0x03 + 0x55 + 0x04 + CRC8;

Slave:

1. For example, unit price: 123.45/litre

0xA5 + 0x01 + 0x03 + 0x04 + 0x00 + 0x01 + 0x23 + 0x45 + CRC8.

2. If information is wrong

0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + CRC8;

4.9 To check the amount of injected fuel & sum of sales each class

(e.g.): Check device address no.01, read function code 0x03, data origin address 0x70, data length 0x0C, CRC8.

Master:

0xA5 + 0x01 + 0x03 + 0x70 + 0x0C + CRC8;

Slave:

1. For example, volume:58.37L; sale:123456.78

0xA5 + 0x01 + 0x03 + 0x0C + 0x00 + 0x00 + 0x00 + 0x58 + 0x37 + 0x00 + 0x00 + 0x00 + 0x12 + 0x34 + 0x56 + 0x78 + CRC8.

2. If information is wrong

0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + CRC8;

4.10 To check the accumulative amount of injected fuel & sum of sales

(e.g.): Check device address no.01, read function code 0x03, data origin address 0x60, data length 0x0C, CRC8.

Master:

0xA5 + 0x01 + 0x03 + 0x60 + 0x0C + CRC8;

Slave:

1. For example, volume:58.37L, sale:123456.78

0xA5 + 0x01 + 0x03 + 0x0C + 0x00 + 0x00 + 0x00 + 0x58 + 0x37 + 0x00 + 0x00 + 0x00 + 0x12 + 0x34 + 0x56 + 0x78 + CRC8.

2. If information is wrong

0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + CRC8;

4.11 To check the accumulative amount of injected fuel & sum of sales and fueling times when device is offline

(e.g.): Check device address no.01, read function code 0x03, data origin address 0x80, data length 0x0E, CRC8.

Master:

0xA5 + 0x01 + 0x03 + 0x80 + 0x0E + CRC8;

Slave:

1. For example, volume:58.37L; sale:123456.78 (offline data)

fueling times: (0000-2000)

0xA5 + 0x01 + 0x03 + 0x0E + 0x00 + 0x00 + 0x00 + 0x58 + 0x37 + 0x00 + 0x00 + 0x00 + 0x12 + 0x34

+0x56 + 0x78 + 0x00 + 0x00 + CRC8.

2. If information is wrong

0xA5 + 0x01 + 0x83 + 0x01 / 0x03 / 0x04/ 0x05/ 0x06 / 0x07 + CRC8;

4.12 Check the fueling record each time when offline.

(e.g.): Check device address no.01, read event function code 0x0C, checking fueling record number (0x0001 + 0x0002 +0x0003 ~ +0x2000), data length 0x08, CRC8.

Master:

0xA5 + 0x01 + 0x0C + 0x00 + 0x01 +0x08 + CRC8.

Slave:

1. For example, The data of fueling of "x" time, volume: 58.37L; sale: 123456.78.

0xA5 + 0x01 + 0x0C + 0x08 + 0x00 + 0x58 + 0x37 + 0x00 + 0x12 + 0x34 + 0x56 + 0x78 + CRC8;

2. If information is wrong

0xA5 + 0x01 + 0x8C + 0x01 / 0x03 / 0x04 / 0x05 / 0x06 / 0x07 + CRC8;

5. Data origin address code:

Sequence	Code	Remark
1	0x00	Device status
2	0x10	Preset value (litre)
3	0x14	Preset value (sale)
4	0x20	Real-time data when device is working
5	0x30	Current data when device stop working Or stop the device during fueling and check current data
6	0x55	Unit price
7	0x5A	Device connecting status "control" or "monitor"
8	0x60	Accumulative total
9	0x70	Total each class
10	0x80	Data bag offline
11	0x83	Read wrong code
12	0x90	Write wrong code
13	0x0C	Event record

6. Error code:

Sequence	Error code	Remark
1	0x01	Wrong command
2	0x03	Device stop working
3	0x04	Resend un-accomplished command
4	0x05	Device is busy
5	0x06	Device is offline
6	0x07	CRC data verify wrong

7. CRC operate mode

code unsigned char code CRC8_TAB[256] =

```
{
    0x00, 0x5e, 0xbc, 0xe2, 0x61, 0x3f, 0xdd, 0x83,
    0xc2, 0x9c, 0x7e, 0x20, 0xa3, 0xfd, 0x1f, 0x41,
    0x9d, 0xc3, 0x21, 0x7f, 0xfc, 0xa2, 0x40, 0x1e,
    0x5f, 0x01, 0xe3, 0xbd, 0x3e, 0x60, 0x82, 0xdc,
    0x23, 0x7d, 0x9f, 0xc1, 0x42, 0x1c, 0xfe, 0xa0,
    0xe1, 0xbf, 0x5d, 0x03, 0x80, 0xde, 0x3c, 0x62,
    0xbe, 0xe0, 0x02, 0x5c, 0xdf, 0x81, 0x63, 0x3d,
    0x7c, 0x22, 0xc0, 0x9e, 0x1d, 0x43, 0xa1, 0xff,
    0x46, 0x18, 0xfa, 0xa4, 0x27, 0x79, 0x9b, 0xc5,
    0x84, 0xda, 0x38, 0x66, 0xe5, 0xbb, 0x59, 0x07,
    0xdb, 0x85, 0x67, 0x39, 0xba, 0xe4, 0x06, 0x58,
    0x19, 0x47, 0xa5, 0xfb, 0x78, 0x26, 0xc4, 0x9a,
    0x65, 0x3b, 0xd9, 0x87, 0x04, 0x5a, 0xb8, 0xe6,
    0xa7, 0xf9, 0x1b, 0x45, 0xc6, 0x98, 0x7a, 0x24,
    0xf8, 0xa6, 0x44, 0x1a, 0x99, 0xc7, 0x25, 0x7b,
    0x3a, 0x64, 0x86, 0xd8, 0x5b, 0x05, 0xe7, 0xb9,
    0x8c, 0xd2, 0x30, 0x6e, 0xed, 0xb3, 0x51, 0x0f,
    0x4e, 0x10, 0xf2, 0xac, 0x2f, 0x71, 0x93, 0xcd,
    0x11, 0x4f, 0xad, 0xf3, 0x70, 0x2e, 0xcc, 0x92,
    0xd3, 0x8d, 0x6f, 0x31, 0xb2, 0xec, 0x0e, 0x50,
    0xaf, 0xf1, 0x13, 0x4d, 0xce, 0x90, 0x72, 0x2c,
    0x6d, 0x33, 0xd1, 0x8f, 0x0c, 0x52, 0xb0, 0xee,
    0x32, 0x6c, 0x8e, 0xd0, 0x53, 0x0d, 0xef, 0xb1,
    0xf0, 0xae, 0x4c, 0x12, 0x91, 0xcf, 0x2d, 0x73,
```

```
0xca, 0x94, 0x76, 0x28, 0xab, 0xf5, 0x17, 0x49,  
0x08, 0x56, 0xb4, 0xea, 0x69, 0x37, 0xd5, 0x8b,  
0x57, 0x09, 0xeb, 0xb5, 0x36, 0x68, 0x8a, 0xd4,  
0x95, 0xcb, 0x29, 0x77, 0xf4, 0xaa, 0x48, 0x16,  
0xe9, 0xb7, 0x55, 0x0b, 0x88, 0xd6, 0x34, 0x6a,  
0x2b, 0x75, 0x97, 0xc9, 0x4a, 0x14, 0xf6, 0xa8,  
0x74, 0x2a, 0xc8, 0x96, 0x15, 0x4b, 0xa9, 0xf7,  
0xb6, 0xe8, 0x0a, 0x54, 0xd7, 0x89, 0x6b, 0x35
```

```
};
```

```
unsigned char CRC8_Tab(unsigned char ucPtr, unsigned char ucLen)  
{ unsigned char ucIndex;  
  unsigned char ucCRC8 = 0;  
  while (ucLen--)  
{ ucIndex = ucCRC8 ^ (ucPtr ++);  
  ucCRC8 = CRC8_TAB[ucIndex]; }  
  return(ucCRC8); }
```